Yenepoya Institute of Arts, Science, Commerce and Management

Project – High Level Design

on

Enterprise Log Analysis for E-Commerce Systems

Student Name: Muhammed Sinan

23BCDSBA24

Industry mentor: Sumit kumar Shukla **Guided By:**Raksha Adyanthaya

Table of Contents

1. Introduction. 3
- 1.1. Scope of the document. 3
- 1.2. Intended Audience 3
- 1.3. System overview. 3

2. System Design. 4
- 2.1. Application Design 4
- 2.2. Process Flow. 4
- 2.3. Information Flow. 5
- 2.4. Components Design 5
- 2.5. Key Design Considerations 6
- 2.6. API Catalogue. 6

3. Data Design. 7
- 3.1. Data Model 7
- 3.2. Data Access Mechanism 8
- 3.3. Data Retention Policies 8
- 3.4. Data Migration 8

4. Interfaces 8

5. State and Session Management 9

6. Caching 10

7. Non-Functional Requirements 10
- 7.1. Security Aspects 11
- 7.2. Performance Aspects 11

8. References 12

1. Introduction

This High Level Design (HLD) document describes the overall architecture and design of the Enterprise Log Analysis for E-Commerce Systems project. The system processes, analyses and visualises raw web server log data collected from an online e-commerce platform using Python-based data science libraries. This document outlines scope, intended audience, system overview, design decisions, data structures, interfaces and non-functional requirements at a high level of abstraction.

1.1. Scope of the document.

This document covers the high-level architecture and design of the entire log analysis pipeline. It describes how raw log files are ingested, processed and transformed into actionable insights through statistical analysis, visualisation and machine learning-based anomaly detection. Implementation-level code logic is addressed in the companion Low Level Design (LLD) document.

1.2. Intended Audience

- Industry mentor and technical reviewers assessing architectural decisions
- College guide and academic examiners evaluating system design completeness
- Developers using this document as a blueprint before implementation
- Project stakeholders requiring a non-code-level understanding of the system

1.3. System overview.

The system ingests three raw Apache-format web server log files — browse, product and basket — sourced from the Harvard Dataverse Online Shopping Store dataset. A multi-stage pipeline extracts timestamp data using Regular Expressions, structures it using Pandas DataFrames, generates three types of visualisation charts using Matplotlib, applies Isolation Forest anomaly detection using Scikit-learn, and produces a formatted summary report. The entire pipeline runs within a Jupyter Notebook environment.

2. System Design.

The system follows a linear pipeline architecture. Each stage is responsible for a single concern — reading, extracting, transforming, visualising or reporting. This separation of concerns ensures that each module is independently testable and replaceable. All state is passed as Python objects between modules.

2.1. Application Design

The application is a sequential data processing pipeline implemented within a Jupyter Notebook, following the Extract-Transform-Load (ETL) pattern adapted for analytical use:

Layer	Role	Technology
Extraction	Read raw log files and parse timestamps using Regex	Python re module
Transformation	Structure data into DataFrames, group by hour and event type	Pandas
Visualisation	Generate bar, line and pie charts from grouped data	Matplotlib
Detection	Identify anomalous traffic hours using unsupervised ML	Scikit-learn (IsolationForest)
Reporting	Compute and print KPIs — totals, peaks, anomalies, conversion rate	Python built-ins

2.2. Process Flow.

The process flow describes the sequential execution order of the pipeline modules. Each module depends on the output of the previous one.

Step	Module	Action	Output
1	Log File Reader	Open and stream log files line by line	Raw text lines
2	Regex Extractor	Match timestamp pattern, extract fields	List of dicts
3	Pandas Handler	Build DataFrame, type-convert, group by hour	3 grouped DataFrames
4	Chart Generator	Generate bar, line and pie charts	3 visualisations

Step	Module	Action	Output
5	Anomaly Detector	Run IsolationForest on browse counts	Anomaly label array
6	Summary Reporter	Compute totals, peaks, conversion rate	Terminal report

2.3. Information Flow.

The table below describes what data object is passed between each pair of modules and in what format it travels through the pipeline.

From	To	Data Passed	Format
Log File Reader	Regex Extractor	Raw log lines, one at a time	Python string (stripped)
Regex Extractor	Pandas Handler	Extracted timestamp fields + event type	Python list of dicts
Pandas Handler	Chart Generator	Grouped hourly event counts per type	Three Pandas DataFrames
Pandas Handler	Anomaly Detector	Browse hourly counts as numeric array	NumPy array (reshaped 2D)
Pandas Handler	Summary Reporter	All grouped DataFrames	Pandas DataFrames
Anomaly Detector	Summary Reporter	Anomaly labels per hour	Pandas Series (-1 / 1)

2.4. Components Design

The system is composed of six discrete components, each corresponding to a logical processing stage implemented as one or more Jupyter Notebook cells.

Component	Responsibility	Input	Output
Log File Reader	Opens and streams log files line by line	File path (string)	Text lines (iterator)
Regex Extractor	Applies pattern matching to extract timestamp fields	Raw text line	Dict per matched line
Pandas Handler	Structures, types and groups extracted data	List of dicts	3 grouped DataFrames
Chart Generator	Produces bar, line and pie visualisations	Grouped DataFrames	3 rendered charts

Component	Responsibility	Input	Output
Anomaly Detector	Runs IsolationForest on browse counts	NumPy array	Anomaly label array
Summary Reporter	Computes and prints all KPIs	All DataFrames + labels	Terminal text output

2.5. Key Design Considerations

- Single-pass file reading: Files are read line-by-line to avoid loading large datasets into memory, making the system suitable for large log files.
- Regex-only extraction: No CSV or JSON parsing is used. The Regex pattern is the sole extraction mechanism, making the system format-sensitive but highly efficient.
- Unsupervised anomaly detection: Isolation Forest requires no labelled training data, appropriate for this exploratory analysis context.
- Separation of concerns: Each module has a single, well-defined role, making individual modules easy to test, replace or extend independently.
- Notebook-native output: All charts and reports render inline in Jupyter, requiring no external dashboard or web server.

2.6. API Catalogue.

The system does not expose external HTTP APIs. The following table documents the internal function-level interface — key Python calls forming the contract between modules.

Module	Key Call / API	Parameters	Returns
Log File Reader	open(path, 'r')	File path string	File object
Regex Extractor	re.match(pattern, line)	Compiled pattern, text line	Match object or None
Pandas Handler	pd.DataFrame(data)	List of dicts	DataFrame
Pandas Handler	df.groupby([...]).size()	Column name list	Grouped Series
Chart Generator	plt.bar() / plt.plot() / plt.pie()	x, y arrays, styling args	Rendered figure
Anomaly Detector	IsolationForest.fit_predict(X)	2D NumPy array	Label array [-1, 1]
Summary Reporter	series.sum() / series.idxmax()	Pandas Series	Scalar int or float

3. Data Design.

This section describes how data is structured, accessed and managed throughout the pipeline. The system operates entirely on in-memory data objects during a single Jupyter Notebook session. No persistent database or file-based storage is used for intermediate results.

3.1. Data Model

The primary data object is a Pandas DataFrame. Raw log lines are first converted to Python dictionaries, aggregated into a single DataFrame, and split into three event-specific DataFrames for downstream use.

Data Object	Type	Fields / Columns	Description
Raw log line	Python string	N/A	Single raw text line from log file, stripped of whitespace
Extracted record	Python dict	day, month, year, hour, event	One parsed record per matched log line
Master DataFrame (df)	Pandas DataFrame	day, month, year, hour, event	All extracted records in a structured table
Grouped DataFrame (hourly)	Pandas DataFrame	hour, event, count	Event counts grouped by hour and event type
browse_data	Pandas DataFrame	hour, count, anomaly	Browse events; anomaly column added by Module 5
product_data	Pandas DataFrame	hour, count	Product events only
basket_data	Pandas DataFrame	hour, count	Basket events only
browse_counts	NumPy ndarray	shape (N, 1)	Browse counts reshaped for Scikit-learn input

3.2. Data Access Mechanism

All data access is in-memory. Files are read using Python's built-in open() context manager in read-only mode. After extraction, all data is held in Pandas DataFrames accessed using standard

Pandas indexing, filtering and groupby operations. No ORM, SQL queries or database connections are used.

Access Type	Mechanism	Usage
File read	open(path, 'r') with for loop	Stream raw log lines one at a time
Pattern matching	re.match(pattern, line)	Extract timestamp fields from each line
DataFrame filtering	df[df["event"] == "browse"]	Separate data by event type
Column access	df["hour"].astype(int)	Type conversion and column selection
Aggregation	df.groupby([...]).size()	Count events per hour and event type
Index lookup	series.idxmax()	Find the index of the peak value

3.3. Data Retention Policies

This system does not persist any data beyond the active Jupyter Notebook session. All DataFrames, arrays and computed metrics exist only in RAM during execution. Upon kernel shutdown, all data is cleared. No logs, results or intermediate files are written to disk. To preserve results, the user must manually export charts using plt.savefig() or DataFrames using df.to_csv().

3.4. Data Migration

There is no data migration requirement for this system. The input data — three static log files from the Harvard Dataverse dataset — are fixed files that do not change between runs. If the dataset is updated or replaced, the only change required is updating the file path strings in the Log File Reader module. No schema migration or database update process is required.

4. Interfaces

The system does not expose any external-facing user interface beyond the Jupyter Notebook environment. All interactions occur within notebook cells. The following interfaces exist within the system:

Interface Type	Description	Direction	Technology
File Interface	System reads three raw log files from local disk	Input (read-only)	Python open()
Notebook Cell Interface	Each module receives inputs from and passes outputs to adjacent cells via shared Python variables	Internal (bidirectional)	Jupyter / Python scope
Chart Display Interface	Matplotlib renders charts inline in the Jupyter output area	Output (display)	Matplotlib / IPython
Terminal Output Interface	Summary Reporter prints KPIs to the Jupyter cell output	Output (print)	Python print()
Scikit-learn Interface	Pandas column passed to IsolationForest as NumPy array; labels returned as Pandas Series	Internal (module-to-module)	Scikit-learn API

5. State and Session Management

The system is stateless between runs. There is no persistent application state, user session, authentication mechanism or cookie management. All state is held in Python variables within the Jupyter Notebook kernel memory for the duration of a single execution session.

State Element	Type	Scope	Lifetime
data (list of dicts)	Python list	Global notebook scope	From extraction cell to DataFrame creation
df (master DataFrame)	Pandas DataFrame	Global notebook scope	From creation until session ends
hourly (grouped DataFrame)	Pandas DataFrame	Global notebook scope	From groupby cell until session ends
browse / product / basket_data	Pandas DataFrames	Global notebook scope	From filter cell until session ends
browse_counts	NumPy array	Global notebook scope	Used only during Module 5 execution
model (IsolationForest)	Sklearn object	Global notebook scope	Fitted once; used for prediction only
anomalies	Pandas DataFrame	Global notebook scope	Derived in Module 5; used in Module 6

Since Jupyter Notebooks maintain a persistent kernel state, all variables remain available to downstream cells. Re-running cells out of order may produce stale state — it is recommended to run all cells sequentially using Run All.

6. Caching

The current system does not implement explicit caching. Since the pipeline reads and processes data within a single Jupyter session, the in-memory Python variables act as an implicit cache — once data is loaded and transformed, the resulting DataFrames are reused across multiple downstream modules without re-reading the source files.

Implicit Cache	How It Works	Benefit
Master DataFrame (df)	Created once in Module 3; reused by all downstream modules	Avoids re-parsing all log files for each chart
Grouped DataFrame (hourly)	Computed once; split into three event DataFrames for reuse	Avoids re-running groupby for each module
IsolationForest model	Fitted once in Module 5; predict() called on same fitted object	Avoids re-training the model for prediction

If extended to production with scheduled log ingestion, a formal caching layer such as Redis, Parquet file storage or Joblib model serialisation would be recommended to avoid re-processing large datasets on each run.

7. Non-Functional Requirements

This section defines the quality attributes and constraints that the system must satisfy beyond its functional behaviour.

7.1. Security Aspect

Requirement	Description	Current Status
File Access Control	Log files are accessed in read-only mode. No write operations are performed on source files.	Implemented — open(path, 'r')
No Network Exposure	The system runs entirely locally within a Jupyter Notebook. No HTTP endpoints, APIs or external connections are opened.	Implemented by design
No Credential Storage	No usernames, passwords or API keys are used or stored anywhere in the system.	Not applicable
Data Privacy	The Harvard Dataverse dataset contains no PII. All records are anonymous web server logs.	Satisfied by dataset choice
Input Validation	The Regex pattern acts as implicit input validation — only lines matching the expected format are processed.	Implemented — if match: guard

7.2. Performance Aspects

Requirement	Description	Design Decision
Memory Efficiency	Files are read line-by-line rather than fully into memory, preventing MemoryError on large datasets.	Streaming file read with for loop
Processing Speed	Pandas vectorised operations are used instead of Python loops for all DataFrame transformations.	Pandas native operations throughout
Scalability	The pipeline handles any dataset size as long as the extracted DataFrame fits in RAM. For very large datasets, chunked reading should be applied.	Line-by-line read; chunked processing recommended for scale
Reproducibility	IsolationForest is initialised with random_state=42, ensuring identical results on every run.	random_state=42 fixed
Execution Time	For the Harvard Dataverse dataset (~5.3M browse events), the full pipeline completes within 30–60 seconds on standard hardware.	Acceptable for batch analysis use case

8. References

Reference	Details
Dataset	Online Shopping Store — Web Server Logs. Harvard Dataverse, 2021. doi:10.7910/DVN/3QBYB5
Python re	Python Software Foundation. re — Regular expression operations. https://docs.python.org/3/library/re.html
Pandas	Pandas Development Team. Pandas Documentation v2.x. https://pandas.pydata.org/docs/
Matplotlib	Matplotlib Development Team. Matplotlib: Visualization with Python. https://matplotlib.org/stable/contents.html
Scikit-learn	Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825–2830, 2011.
Isolation Forest	Liu, Ting and Zhou. Isolation Forest. IEEE ICDM, 2008.
Jupyter Notebook	Project Jupyter. Jupyter Notebook Documentation. https://jupyter-notebook.readthedocs.io/
Harvard Dataverse	Harvard Dataverse. Open Data Repository. https://dataverse.harvard.edu/